

SHACASDAG

*A SHA-Anchored Content-Addressable Storage / Directed
Acyclic Graph
Constitutional Kernel*

Author: John — AETHEL Systems Research

Date: July 2026

Version 1.0-DRAFT

Abstract

Distributed systems at scale face a fundamental gap between the guarantees offered by formal verification frameworks and the ad hoc invariant management found in production deployments. State transitions in contemporary distributed kernels are neither content-addressed nor causally structured in a manner amenable to machine-checkable constitutional enforcement. As a result, invariant violations surface only at runtime, often under adversarial conditions, with no principled recovery path.

We present **SHACASDAG** — a SHA-Anchored Content-Addressable Storage / Directed Acyclic Graph Constitutional Kernel — a formally grounded architecture in which every state transition is encoded as a SHA-256-hashed CAS node, causal dependency is expressed as edges in a Directed Acyclic Graph, and constitutional invariants are encoded as TLA+ predicates evaluated at each AETHEL Gate admission point. The kernel guarantees that no state may be appended to the DAG unless it has been verified for hash integrity, passes all registered gate

preconditions and postconditions, and (in the multinode setting) carries a Byzantine-fault-tolerant Quorum Certificate from the validator set.

The principal contributions of this work are: (1) a rigorous formal definition hierarchy establishing the CAS node, DAG, constitutional invariant, AETHEL Gate, and Quorum Certificate as first-class mathematical objects; (2) a theorem hierarchy proving SHA integrity, DAG acyclicity preservation, constitutional invariant preservation (safety), liveness under quorum, fork safety, kernel minimality, and deterministic replay; (3) a Rust reference implementation providing a no-unsafe constitutional kernel core with a trait-based gate abstraction, RocksDB-backed persistent DAG storage, and Ed25519 signature verification; and (4) a Byzantine-fault-tolerant multinode extension, AETHEL-BFT, together with a sharding proposal (SHACASDAG-S) for horizontal scalability. The design is grounded in the AETHEL Gate Pipeline TLA+ formal specification, providing an executable formal model against which the implementation can be model-checked.

Keywords: content-addressable storage, directed acyclic graph, constitutional kernel, TLA+, Byzantine fault tolerance, SHA-256, formal verification, Rust, distributed systems.

Table of Contents

Abstract

1. Introduction

1.1 Motivation

1.2 The Constitutional Kernel Concept

1.3 Scope and Delegation

1.4 Prior Work

2. Formal Foundations

2.1 Core Definitions

2.2 Theorem Hierarchy

3. TLA+ Specification (AETHEL Gate Pipeline)

3.1 State Variables

3.2 Initial State

3.3 Gate Admission Action

3.4 Quorum Commit Action

3.5 Key Invariants

4. Rust Architecture

4.1 Design Philosophy

4.2 Core Type System

4.3 Error Types

4.4 Crate Dependency Graph

5. Multinode Extension

5.1 Network Model

5.2 Node Roles

5.3 Consensus Protocol (AETHEL-BFT)

5.4 View Change

5.5 DAG Sync Protocol

5.6 Sharding Extension (SHACASDAG-S)

6. Constitutional Governance Model

6.1 Gate Registry

6.2 Invariant Upgrades

6.3 Validator Set Management

7. Security Analysis

7.1 Threat Model

7.2 SHA-256 Collision Resistance

7.3 Signature Scheme

7.4 Known Limitations

8. Implementation Roadmap

9. Conclusion

References

1. Introduction

1.1 Motivation

The formal verification community has produced powerful tools — TLA+ [1], Coq [15], Isabelle/HOL — capable of specifying and model-checking complex distributed protocols. Yet the gap between a verified specification and a production distributed system remains wide. State machines in deployed systems typically lack a canonical content address for each state, offer no machine-checkable causal ordering among transitions, and enforce invariants only through procedural guards that are invisible to any model checker. When a node crashes, a network partitions, or a Byzantine actor injects a malformed message, there is no structural mechanism to guarantee that invariants are re-established on recovery.

The shortcoming is not merely academic. Production distributed databases, consensus engines, and coordination services regularly surface invariant violations only in post-incident analysis. The standard response — "add a check" — is informal, untested against the formal model, and irreproducible across deployments. What is needed is a constitutional layer: a minimal, formally specified, invariant-enforcing core that sits beneath application logic and guarantees, by construction, that every state transition is causally ordered, content-authenticated, and predicate-verified.

SHACASDAG addresses this need by treating each state transition as a content-addressable node in a Directed Acyclic Graph, whose admission to the DAG is gated by a formally specified predicate hierarchy — the AETHEL Gate Pipeline — expressed in TLA+. The result is a system in which the formal specification is not merely documentation but is operationally enforced at every write.

1.2 The Constitutional Kernel Concept

The term *constitutional kernel* is used deliberately to evoke the analogy with constitutional law. A constitution does not implement the functions of government; it specifies the invariants that all government actions must preserve, the procedures by which new laws may be admitted, and the minimal structure that cannot be abrogated. Similarly, the SHACASDAG constitutional kernel does not implement application logic; it specifies and enforces the invariants that all state transitions must preserve, the gate admission procedure by which new transitions are accepted, and a minimal, formally verified core that cannot be bypassed.

This design principle draws directly from the seL4 microkernel [5], which demonstrated that a formally verified, minimal kernel can provide ironclad guarantees to higher layers without dictating their policy. The constitutional kernel analogue for distributed systems is SHACASDAG: a minimal core providing content integrity, causal ordering, invariant preservation, and

Byzantine-fault-tolerant consensus, upon which arbitrary application state machines may be composed.

1.3 Scope and Delegation

SHACASDAG governs the following concerns exclusively:

- **State transition admission:** A transition from state u to state v is admitted if and only if it passes hash verification and all registered AETHEL Gates.
- **Causal ordering:** The DAG structure encodes happens-before relationships; no node may reference a parent not already present in the DAG.
- **Constitutional invariant preservation:** The predicate I must hold at every committed node; gates are the only mechanism by which I can be checked and transitions can alter state.
- **Quorum consensus:** In the multinode setting, a node is committed only upon receipt of a Quorum Certificate satisfying the Byzantine threshold.

SHACASDAG explicitly *delegates* the following to application layers: the semantic interpretation of payload content, application-level business logic, network transport selection, query routing, and any concern not directly bearing on state transition integrity. This tight scope is the source of the kernel's formal tractability.

1.4 Prior Work

SHACASDAG synthesizes ideas from several lines of prior work. Content-addressable storage was systematized by the InterPlanetary File System (IPFS) [4], which demonstrated the practical utility of SHA-based node identity for distributed data. Git's Merkle DAG [3] established the DAG-of-content-addressed-commits as a practical version control structure,

providing a template for causal ordering via hash-linked parents. Nakamoto's Bitcoin [6] introduced the proof-of-work blockchain as a globally ordered DAG with Byzantine fault tolerance; SHACASDAG replaces proof-of-work with a BFT quorum mechanism.

Byzantine fault-tolerant consensus was placed on firm practical footing by Castro and Liskov's PBFT [2], whose three-phase protocol and quorum intersection arguments underpin the AETHEL-BFT consensus protocol described in Section 5. DAG-based consensus designs have more recently been explored in IOTA's Tangle [7] and Avalanche [8], both of which motivate the DAG structure of SHACASDAG while differing in their fault models and confirmation mechanisms.

On the formal verification side, Lamport's TLA+ [1] provides the specification language for the AETHEL Gate Pipeline, while the seL4 project [5] provides the existence proof that a formally verified kernel can be deployed in production environments. The partial synchrony model of Dwork, Lynch, and Stockmeyer [14] underpins the liveness analysis of AETHEL-BFT. The Ed25519 signature scheme [10] and SHA-256 hash function [9] provide the cryptographic primitives, and BLS12-381 [13] is identified as a future upgrade path for signature aggregation. The Rust ownership model [11] provides memory safety guarantees for the reference implementation without requiring a garbage collector.

2. Formal Foundations

2.1 Core Definitions

We present the formal definitional hierarchy of SHACASDAG. Definitions are stated in set-theoretic notation augmented with TLA+ predicate syntax where relevant. All hash references are over the SHA-256 function, denoted $\text{SHA-256}(\cdot)$. Canonical encoding of a structured value x is denoted

`canonical_encode(x)` and is defined as the deterministic binary serialization produced by the kernel's canonical codec (bincode in the Rust reference implementation, with field ordering fixed by schema version).

Definition 1 — CAS Node

A **CAS Node** N is a tuple $N = (h, \text{payload}, \text{meta})$ where:

- $h = \text{SHA-256}(\text{canonical_encode}(\text{payload} \parallel \text{meta}))$ — the content address (a 256-bit digest);
- $\text{payload} \in \Sigma^*$ — an arbitrary finite byte string representing the transition payload;
- $\text{meta} = \{ \text{type}: \text{NodeType}, \text{ts}: \mathbb{N}, \text{author}: \text{PubKey}, \text{parents}: \text{List}[\text{Hash}] \}$ — a fixed-schema metadata record;
- $\text{NodeType} \in \{ \text{Genesis}, \text{Transition}, \text{ValidatorSetChange}, \text{GovernanceProposal}, \text{Checkpoint} \}$.

Integrity invariant: $\forall N. \text{verify}(N) \Leftrightarrow h = \text{SHA-256}(\text{canonical_encode}(\text{payload} \parallel \text{meta}))$.

The verify predicate is the sole gate by which a byte string is admitted as a structurally valid CAS node. Any node for which $\text{verify}(N) = \text{false}$ is rejected prior to DAG insertion and prior to gate evaluation.

Definition 2 — SHACASDAG (Directed Acyclic Graph)

A **SHACASDAG** $G = (V, E)$ is a directed graph where:

- $V \subseteq \{ N \mid \text{verify}(N) = \text{true} \}$ — the vertex set, consisting exclusively of verified CAS nodes;

- $E \subseteq V \times V$ — directed edges $(u \rightarrow v)$, where $(u, v) \in E$ iff $u.\text{hash} \in v.\text{meta}.\text{parents}$, encoding causal precedence;
- **Acyclicity invariant:** $\forall$ paths P in G . P is finite and non-repeating (no vertex appears twice);
- **Genesis node** G_0 : the unique node with $G_0.\text{meta}.\text{parents} = \emptyset$ and $\text{NodeType} = \text{Genesis}$. G is rooted at G_0 .

The height of a node v , denoted $\text{height}(v)$, is the length of the longest path from G_0 to v . $\text{height}(G_0) = 0$.

Definition 3 — Constitutional Invariant

A **Constitutional Invariant** is a predicate $I: V \rightarrow \{\text{true}, \text{false}\}$ such that:

- $I(G_0) = \text{true}$ — the invariant holds at the genesis node;
- $\forall (u, v) \in E: I(u) = \text{true} \wedge \text{GateCheck}(u, v) \Rightarrow I(v) = \text{true}$ — the invariant is preserved across every gate-admitted transition.

I is expressed as a composable predicate tree, permitting conjunctive extension via governance proposals (see Section 6.2). The primitive atomic predicates of I are registered at genesis and may be versioned via the gate manifest.

Definition 4 — AETHEL Gate

An **AETHEL Gate** $\Gamma = (\text{precond}, \text{postcond}, \text{effect})$ is a triple where:

- $\text{precond}: \text{State} \rightarrow \text{Bool}$ — a TLA+ safety predicate evaluated over the

state of the source node;

- *postcond*: $\text{State} \rightarrow \text{Bool}$ — a TLA+ liveness predicate evaluated over the state of the destination node;
- *effect*: $\text{State} \rightarrow \text{State}$ — a deterministic, total state transformer encoding the transition semantics.

Admission rule: Gate Γ *admits* transition $(u \rightarrow v)$ iff:

- $\text{precond}(\text{state}(u)) = \text{true}$,
- $\text{verify_hash}(v) = \text{true}$, and
- $\text{postcond}(\text{effect}(\text{state}(u))) = \text{true}$.

A gate must be pure: *effect* has no side effects observable outside the kernel state. Gates are registered in the gate manifest at genesis and may only be modified via the governance protocol.

Definition 5 — Constitutional Kernel

The **Constitutional Kernel** $K = (G, \Gamma^*, I)$ consists of:

- G : a valid SHACASDAG (Definition 2);
- Γ^* : the ordered set of all registered AETHEL Gates;
- I : the constitutional invariant (Definition 3).

K is **valid** iff:

- G is a valid SHACASDAG;
- $\forall$ transitions $(u, v) \in E: \exists \Gamma \in \Gamma^*$ such that Γ admits $(u \rightarrow v)$;
- I holds at every node in V .

The kernel enforces these three conditions by construction at every write path; no circumvention is structurally possible without producing a hash mismatch detectable by any replica.

Definition 6 — Quorum Certificate (QC)

A **Quorum Certificate** QC for node N is a tuple $QC = (node_hash, view, signatures)$ where:

- $node_hash = h(N)$ — the content address of the certified node;
- $view \in \mathbb{N}$ — the consensus view number at which the certificate was formed;
- $signatures = \{\sigma_i\}$ — a set of $n\hat{Z}$ valid Ed25519 signatures from distinct validators in the current validator set, where $n\hat{Z} \geq \lceil (2n+1)/3 \rceil$ and each $\sigma_i = \text{Sign}(sk_i, h(N))$.

$is_valid(QC, \text{Validators}, \text{threshold}) = \text{true}$ iff every $\sigma_i \in signatures$ verifies against a distinct $pk_i \in \text{Validators}$ and $|signatures| \geq \text{threshold}$.

2.2 Theorem Hierarchy

We establish a hierarchy of theorems, lemmas, and corollaries characterizing the safety, liveness, and structural properties of the SHACASDAG constitutional kernel. Proof sketches are provided; complete formal proofs are targets of the TLA+ model-check and Prusti annotation passes scheduled for Phase 5 (Section 8).

Theorem 1 — SHA Integrity (Content Authenticity)

For any $N \in V$, any modification to payload or meta produces a hash $h' \neq h$ with probability at least $1 - 2^{-128}$, rendering the modified object inadmissible to V .

Proof sketch. By the collision resistance of SHA-256 under the random oracle model and the standard computational hardness assumption. For any two distinct inputs $x \neq x'$, $\Pr[\text{SHA-256}(x) = \text{SHA-256}(x')] \leq 2^{-128}$ (birthday bound over the 256-bit output space). Let δ be any non-trivial perturbation to $(\text{payload} \parallel \text{meta})$. Then $\text{canonical_encode}(\text{payload} \parallel \text{meta}) \neq \text{canonical_encode}((\text{payload} + \delta) \parallel \text{meta})$, so SHA-256 of the two encodings differ with probability $1 - 2^{-128}$. The vertex set V admits only nodes satisfying $\text{verify}(N)$; therefore the modified node is rejected. $\square$

Theorem 2 — DAG Acyclicity Preservation

No sequence of GateCheck-admitted transitions can introduce a cycle into G .

Proof sketch. By structural induction on $\text{height}(G)$. **Base:** G_0 is acyclic (single node, no edges). **Step:** Suppose G at height k is acyclic. A new node v at height $k+1$ may only reference nodes already present in G as its parents (enforced by the MissingParent check in `DagStore::insert`). The content address $h(v)$ is a deterministic function of $(\text{payload}(v) \parallel \text{meta}(v))$, which includes the set of parent hashes. For v to introduce a cycle, some ancestor u of v would require $h(v) \in u.\text{meta.parents}$, implying $h(v)$ was known before v was constructed — a contradiction with SHA-256 preimage resistance. $\square$

Theorem 3 — Constitutional Invariant Preservation (Safety)

If $I(G_0) = \text{true}$ and every edge $(u, v) \in E$ passes *GateCheck*, then $\forall v \in V: I(v) = \text{true}$.

Proof sketch. By structural induction on G . **Base:** $I(G_0) = \text{true}$ by hypothesis (the genesis node is constructed to satisfy all invariants). **Step:** Assume $I(u) = \text{true}$ for all parents u of v . By Definition 4, *GateCheck*(u, v) requires $\text{precond}(\text{state}(u)) = \text{true}$ and $\text{postcond}(\text{effect}(\text{state}(u))) = \text{true}$. By Lemma 3.1, $\text{postcond}(v) = \text{true}$. By gate design, *postcond* is defined as a superset of the atomic predicates of I ; therefore $I(v) = \text{true}$. $\square$

Lemma 3.1 — Gate Soundness

If $\text{GateCheck}(u, v) = \text{true}$ then $\text{postcond}(\text{state}(v)) = \text{true}$.

Proof. Direct from Definition 4. *GateCheck* evaluates $\text{postcond}(\text{effect}(\text{state}(u)))$ and admits the transition only if this is true. Since *effect* is deterministic and $\text{state}(v) = \text{effect}(\text{state}(u))$ by the transition semantics, $\text{postcond}(\text{state}(v)) = \text{true}$ follows immediately. $\square$

Theorem 4 — Liveness Under Quorum (Progress)

In a network of n validators with at most $f < n/3$ Byzantine faults, any valid proposed node possessing a QC will eventually be appended to G on all honest nodes.

Proof sketch. Follows from the quorum intersection property of PBFT-style BFT consensus [2]. Any two quorums of size $\lceil (2n+1)/3 \rceil$ share at least one honest node (since $2 \cdot \lceil (2n+1)/3 \rceil > n + f$, and the f Byzantine nodes cannot

fill the intersection). An honest node votes only for nodes passing GateCheck (Lemma 3.1) and hash verification (Theorem 1). Therefore a QC attests that at least one honest node admitted the node, guaranteeing invariant preservation. After GST [14], honest validators make progress and the node is appended within a bounded number of message rounds. $\square$

Corollary 4.1 — Fork Safety

No two conflicting nodes (same parent set, different payloads) can both obtain a valid QC in the same view.

Proof sketch. Suppose conflicting nodes N and N' (same parents, $\text{payload}(N) \neq \text{payload}(N')$) both obtain QCs in view v . Each QC requires $\lfloor (2n+1)/3 \rfloor$ signatures from distinct validators. By quorum intersection, at least one honest validator signed both — but an honest validator signs at most one node per parent set per view. Contradiction. $\square$

Theorem 5 — Kernel Minimality

Removing any component of $K = (G, \Gamma^, I)$ breaks at least one of: content integrity, invariant preservation, or progress.*

Proof sketch. By contradiction for each component. (i) Removing G : without the DAG structure and hash-linked parents, causal ordering cannot be established and Theorem 2 fails; cycles become constructible. (ii) Removing Γ^* : without gate admission, arbitrary transitions are accepted, breaking Theorem 3. (iii) Removing I : without a constitutional invariant, gate postconditions have no fixed-point target, and the

soundness guarantee of Lemma 3.1 is vacuous. $\square$

Theorem 6 — Deterministic Replay

Any prefix of G can be deterministically replayed from G_0 by any node possessing the genesis configuration and the DAG node set.

Proof sketch. The content address of each node encodes its full causal history (payload, meta, parents). The topological ordering of G is unique up to the partial order induced by E ; any linear extension of this partial order produces the same sequence of kernel states when each gate's deterministic *effect* function is applied in order. Since *effect* is pure and deterministic (Definition 4), and canonical encoding is bijective, the replay produces identical states regardless of which honest replica performs it.

$\square$

3. TLA+ Specification (AETHEL Gate Pipeline)

The AETHEL Gate Pipeline is the formal specification grounding the SHACASDAG constitutional kernel. It is written in TLA+ and is intended to be machine-checked with TLC or Apalache. The specification models a single-shard kernel; the multinode and sharding extensions are addressed in Section 5. We present the specification in narrative form with inline TLA+ pseudocode blocks.

3.1 State Variables

The specification models five state variables capturing the complete observable state of the kernel at any point in time:

```

VARIABLES    dag,          \* Set of verified CAS nodes currently in G
pending,     \* Set of proposed nodes awaiting gate evaluation    votes,
\* Mapping: NodeHash -> Set of validator signatures    committed,    \* Set of
nodes with a valid Quorum Certificate    view          \* Current consensus
view number (natural number)

```

The `dag` variable represents the authoritative vertex set V of the SHACASDAG. `pending` holds nodes that have been broadcast by a proposer but not yet admitted through any gate. `votes` accumulates validator signatures keyed by node hash. `committed` is the subset of `dag` for which a QC has been assembled and verified. `view` is monotonically non-decreasing and increments on view change.

3.2 Initial State

The initial state places the system at genesis, with only the genesis node committed and all other collections empty:

```

Init ==    /\ dag = {Genesis}    /\ pending = {}    /\ votes = [h \in {} |-> {}]
/\ committed = {Genesis}    /\ view = 0

```

The `Genesis` constant is the unique node satisfying `NodeType = Genesis` and `parents = ∅`. Its hash is fixed in the genesis configuration and is distributed out-of-band to all validators. The genesis node is pre-committed by convention; it requires no QC because it has no causal predecessors to conflict with.

3.3 Gate Admission Action

The `GateAdmit` action models the admission of a pending node into the DAG after successful gate evaluation:

```

GateAdmit(n) ==    /\ n \in pending    /\ \E gate \in Gates :    /\
gate.precond(ParentState(n))    /\ VerifyHash(n)    /\
gate.postcond(ApplyEffect(gate, n))    /\ dag' = dag \cup {n}    /\ pending' =
pending \ {n}    /\ UNCHANGED <<votes, committed, view>>

```

The existential quantification over `Gates` captures the first-admitted-wins semantics of the gate set: a node is admitted if any registered gate admits it. `ParentState(n)` computes the kernel state at the node's most recent parent (or the genesis state if `n`'s parents include only the genesis node).

`VerifyHash(n)` implements the SHA-256 integrity check of Definition 1. `ApplyEffect(gate, n)` applies the gate's deterministic effect function.

3.4 Quorum Commit Action

The `QuorumCommit` action advances a DAG-admitted node to committed status upon accumulation of a threshold number of validator votes:

```
QuorumCommit(n) == /\ n \in dag /\ Cardinality(votes[Hash(n)]) >= Quorum
/\ committed' = committed \cup {n} /\ UNCHANGED <<dag, pending, votes,
view>>
```

The `Quorum` constant is defined as $\lceil (2n+1)/3 \rceil$ where n is the `Cardinality(Validators)` constant. The votes set for a given hash is populated by a separate `CastVote` action (omitted for brevity) that admits a validator's signature only if: (a) the validator is in the current validator set, (b) the signature verifies against the validator's registered public key, and (c) the validator has not already cast a vote for a conflicting node in the current view.

3.5 Key Invariants (TLA+ Style)

The following invariants are stated as TLA+ predicates and are checked by TLC over finite instantiations of the model:

```
TypeInvariant == /\ dag \subseteq ValidNodes /\ committed \subseteq dag
/\ \A n \in dag : InvariantI(n) SafetyInvariant == \A n \in committed : \A
m \in committed : Conflicts(n, m) => FALSE LivenessProperty == \A n \
in dag : []<>(n \in committed)
```

`TypeInvariant` asserts that the dag contains only structurally valid nodes, that committed is a subset of dag (no node can be committed before being admitted), and that the constitutional invariant I holds at every DAG node. `SafetyInvariant` asserts the absence of committed conflicts (Corollary 4.1). `LivenessProperty` is a temporal property asserting that every admitted node is eventually committed, corresponding to Theorem 4. Under the partial synchrony model, `LivenessProperty` holds after GST; the TLA+ model is augmented with a fairness assumption encoding GST.

Note: TLA+ Model Scope

The TLA+ specification targets finite instantiations suitable for TLC model checking (typically $n \leq 5$ validators, DAG depth ≤ 8). Apache's symbolic model checker is targeted for larger parameter spaces. Full mechanization is a Phase 5 deliverable (Section 8).

4. Rust Architecture

4.1 Design Philosophy

The Rust reference implementation of the SHACASDAG constitutional kernel is designed around four non-negotiable principles:

1. **No unsafe code in the kernel core.** The constitutional kernel crate (`shacasdag-kernel`) is compiled with `#![forbid(unsafe_code)]`. Memory safety is provided by the Rust ownership model [11] without requiring runtime garbage collection or unsafe pointer arithmetic.
2. **Trait-based gate abstraction.** The `Gate` trait provides a stable interface for extensible gate registration. Application-level gates implement this trait and are registered at kernel genesis; the kernel core has no compile-time knowledge of application semantics.
3. **Async-first I/O.** Network operations and storage I/O are implemented using the Tokio async runtime, enabling high-concurrency node propagation without blocking the gate evaluation path.
4. **Canonical determinism.** All serialization uses bincode with schema-pinned field ordering, ensuring that `canonical_encode` is deterministic across architectures, compiler versions, and deployments.

4.2 Core Type System

CAS Node

The `CasNode` struct implements Definition 1. The constructor computes the hash at construction time; the `verify` method re-derives it on demand for integrity checking:

```
use sha2::{Sha256, Digest}; use serde::{Serialize, Deserialize}; pub type
Hash = [u8; 32]; #[derive(Clone, Debug, Serialize, Deserialize)] pub struct
NodeMeta { pub node_type: NodeType, pub timestamp: u64, pub
author: ed25519_dalek::PublicKey, pub parents: Vec<Hash>, }
#[derive(Clone, Debug, Serialize, Deserialize)] pub struct CasNode { pub
hash: Hash, pub payload: Vec<u8>, pub meta: NodeMeta, } impl
CasNode { // Construct a new CAS node, computing its content address.
pub fn new(payload: Vec<u8>, meta: NodeMeta) -> Self { let mut hasher
= Sha256::new(); hasher.update(&payload);
hasher.update(&bincode::serialize(&meta).unwrap()); let hash: Hash =
hasher.finalize().into(); CasNode { hash, payload,
meta } } // Verify that the stored hash matches the recomputed
hash. // This is the gate admission prerequisite for every node. pub
fn verify(&self) -> bool { let mut hasher = Sha256::new();
hasher.update(&self.payload);
hasher.update(&bincode::serialize(&self.meta).unwrap()); let
expected: Hash = hasher.finalize().into(); self.hash ==
expected } }
```

Gate Trait

The `Gate` trait encapsulates Definition 4. The `admit` default method composes verification, precondition, effect, and postcondition evaluation into a single transactional check, returning `Some(next_state)` on admission or `None` on rejection:

```
pub trait Gate: Send + Sync { // Unique identifier for this gate, used
in the gate manifest. fn id(&self) -> &str; // TLA+ safety
predicate: must hold over the source state. fn precondition(&self, state:
&KernelState) -> bool; // TLA+ liveness predicate: must hold over the
post-transition state. fn postcondition(&self, state: &KernelState, node:
&CasNode) -> bool; // Deterministic state transformer. Must be pure (no
side effects). fn effect(&self, state: KernelState, node: &CasNode) ->
KernelState; // Composed admission check. Returns the next state on
success. fn admit(&self, state: &KernelState, node: &CasNode) ->
Option<KernelState> { if !node.verify() { return None; }
if !self.precondition(state) { return None; } let next =
self.effect(state.clone(), node); if self.postcondition(&next, node)
{ Some(next) } else { None } }
```

DAG Store

The `DagStore` provides persistent DAG storage over RocksDB with in-memory tip tracking and an async insertion path that enforces parent availability before gate evaluation:

```
pub struct DagStore { db: rocksdb::DB, tips:
Arc<RwLock<HashSet<Hash>>>, gates: Vec<Box<dyn Gate>>, } impl DagStore
{ // Insert a node into the DAG, running full gate admission. //
Returns the new kernel state on success, or a KernelError. pub async fn
insert(&self, node: CasNode, state: &KernelState,
) -> Result<KernelState, KernelError> { // Step 1: Hash integrity
check (Theorem 1) if !node.verify() { return
Err(KernelError::HashMismatch); } // Step 2: Parent
availability check (Theorem 2) for parent in &node.meta.parents {
if !self.contains(parent) { return
Err(KernelError::MissingParent(*parent)); } } //
Step 3: Gate admission (Definition 4, Lemma 3.1) for gate in
&self.gates { if let Some(next_state) = gate.admit(state, &node)
{ self.db.put(node.hash, bincode::serialize(&node)?);
self.update_tips(&node); return Ok(next_state); }
} Err(KernelError::NoGateAdmitted) } // Returns an iterator
over all nodes in topological order (BFS from genesis). pub fn
topological_iter(&self) -> impl Iterator<Item = CasNode>
{ TopoIter::new(&self.db, self.genesis_hash()) } }
```

Quorum Certificate

The `QuorumCertificate` struct implements Definition 6. The `is_valid` method verifies each signature against the registered validator set and enforces the threshold:

```
#[derive(Clone, Debug, Serialize, Deserialize)] pub struct QuorumCertificate
{ pub node_hash: Hash, pub view: u64, pub signatures:
Vec<(ed25519_dalek::PublicKey, ed25519_dalek::Signature)>, } impl
QuorumCertificate { // Validate the QC against the current validator set
and threshold. pub fn is_valid(&self, validators:
&[ed25519_dalek::PublicKey], threshold: usize, ) -> bool {
let valid_sigs = self.signatures.iter().filter(|(pk, sig)| {
validators.contains(pk) && pk.verify(&self.node_hash,
sig).is_ok() })
.count(); valid_sigs >=
threshold } }
```

Constitutional Kernel

The `ConstitutionalKernel` is the top-level entry point, composing the DAG store, kernel state, validator set, and quorum threshold into a single coherent interface. The `replay_from_genesis` method implements Theorem 6:

```

pub struct ConstitutionalKernel {    dag:        Arc<DagStore>,    state:
Arc<RwLock<KernelState>>,    validators: Vec<ed25519_dalek::PublicKey>,
quorum_n:    usize,    // = ceil((2n+1)/3) } impl ConstitutionalKernel
{    /// Propose a new node for gate evaluation and DAG insertion.    pub
async fn propose(&self, node: CasNode) -> Result<(), KernelError>
{    let state = self.state.read().await.clone();    let next =
self.dag.insert(node, &state).await?;    *self.state.write().await =
next;    Ok(())    }    /// Commit a node that has accumulated a
valid Quorum Certificate.    pub async fn commit(&self, qc:
QuorumCertificate) -> Result<(), KernelError> {    if !
qc.is_valid(&self.validators, self.quorum_n) {    return
Err(KernelError::InvalidQC);    }
self.dag.mark_committed(qc.node_hash).await?;    Ok(())    }    ///
Deterministically replay all states from genesis (Theorem 6).    pub fn
replay_from_genesis(&self) -> impl Iterator<Item = KernelState> + '_ {
self.dag.topological_iter().scan(KernelState::genesis(), |s, node| {
for gate in self.dag.gates() {    if let Some(next) =
gate.admit(s, &node) {    *s = next.clone();
return Some(next);    }    }
None    })    } }

```

4.3 Error Types

All kernel error conditions are expressed as a typed enum using `thiserror`, ensuring that every failure mode is explicitly named and documentable at the API boundary:

```

#[derive(Debug, thiserror::Error)] pub enum KernelError {    #[error("SHA-
256 hash mismatch: node integrity violated")]    HashMismatch,
#[error("Parent node {0:?} not found in DAG")]    MissingParent(Hash),
#[error("No registered gate admitted this node")]    NoGateAdmitted,
#[error("Quorum certificate invalid or insufficient signatures")]
InvalidQC,    #[error("Storage error: {0}")]    Storage(#[from]
rocksdb::Error),    #[error("Serialization error: {0}")]
Serialization(#[from] bincode::Error), }

```

4.4 Crate Dependency Graph

The following table enumerates the external crate dependencies of the `shacasdag-kernel` crate and their roles in the implementation:

Crate	Version (Target)	Role
<code>sha2</code>	0.10+	SHA-256 content addressing; the sole hash primitive in the kernel core.
<code>ed25519-dalek</code>	2.0+	Validator signatures and QC verification; provides Sign and Verify

Crate	Version (Target)	Role
		operations.
rocksdb	0.21+	Persistent DAG node storage; LSM-tree structure suitable for append-heavy workloads.
tokio	1.x	Async runtime for network I/O and concurrent insert/commit operations.
bincode	2.0+	Canonical binary serialization for hash pre-image construction; schema-pinned for determinism.
serde	1.x	Serialization trait derivation for CasNode, NodeMeta, QC, and KernelState.
thiserror	1.x	Ergonomic error type definitions for KernelError; zero-cost at runtime.
tonic	0.11+	gRPC transport for peer-to-peer node propagation in the multinode extension.

All dependencies are pinned in `Cargo.lock` for reproducible builds. The `sha2`, `ed25519-dalek`, and `bincode` crates are considered security-sensitive and are subject to additional audit review in the Phase 5 formal verification pass.

5. Multinode Extension

5.1 Network Model

The multinode extension of SHACASDAG operates under the **partial synchrony** network model of Dwork, Lynch, and Stockmeyer [14]. In this model, a Global Stabilization Time (GST) exists after which all messages

between honest nodes are delivered within a known bound Δ . Prior to GST, the network may be asynchronous: messages may be delayed, reordered, or dropped. Safety (Theorem 3, Corollary 4.1) is guaranteed unconditionally (before and after GST). Liveness (Theorem 4) is guaranteed only after GST.

The system comprises n validator nodes, of which at most $f < n/3$ are Byzantine. Byzantine nodes may deviate arbitrarily from the protocol: they may send conflicting messages, equivocate, delay messages, or remain silent. Honest nodes follow the AETHEL-BFT protocol exactly. All inter-node messages are signed with the sender's Ed25519 key and are hash-addressed, enabling any recipient to verify both authorship and message integrity.

5.2 Node Roles

Role	Responsibilities	Voting Rights
Proposer	Constructs a CasNode from pending application transitions; broadcasts it to all validators with full payload and meta.	No (may also be a Validator)
Validator	Verifies hash integrity (Theorem 1); evaluates GateCheck (Definition 4); broadcasts a signed VOTE if admitted.	Yes
Aggregator	Collects VOTE messages; assembles a QuorumCertificate once the threshold is reached; broadcasts the QC to all nodes.	No (may overlap with Validator)
Observer	Maintains a read-only DAG replica; receives committed nodes and QCs; serves DAG queries to clients.	No

In a minimal deployment, all roles may be co-located on a single process per validator node. In larger deployments, the Aggregator role may be rotated

among validators or outsourced to a dedicated aggregation service, provided the QC is still verified by each node before marking a node committed.

5.3 Consensus Protocol (AETHEL-BFT)

AETHEL-BFT is a three-phase Byzantine fault-tolerant consensus protocol adapted to the SHACASDAG DAG structure. It is inspired by PBFT [2] but operates natively over content-addressed nodes rather than sequential log entries.

Phase 1 — PROPOSE

The designated Proposer for the current view constructs a `CasNode` N with payload, meta (including parent hashes referencing the current DAG tips), and computes $h(N)$. The Proposer broadcasts a PROPOSE message to all validators:

```
PROPOSE(view, N) -- signed by Proposer's key   where N = CasNode { hash,
payload, meta }
```

Phase 2 — VOTE

Upon receiving PROPOSE($view$, N), each validator:

5. Verifies the Proposer's signature on the PROPOSE message.
6. Checks that $view$ equals the validator's current view.
7. Verifies hash integrity: $h(N)$ must equal SHA-256(canonical_encode(payload || meta)).
8. Verifies that all parent hashes in N .meta.parents are present in the validator's local DAG.
9. Evaluates GateCheck($parent_state$, N) for each registered gate.
10. If all checks pass, broadcasts a VOTE message signed with the validator's Ed25519 key:

```
VOTE(view, Hash(N), sigma_i) -- sigma_i = Sign(sk_i, Hash(N))
```


If any check fails, the validator does not vote and may optionally broadcast a REJECT message for diagnostic purposes.

Phase 3 — COMMIT

The Aggregator collects VOTE messages for $\text{Hash}(M)$ in view $view$. Once $\lceil (2n+1)/3 \rceil$ distinct valid votes are collected, the Aggregator assembles a QuorumCertificate and broadcasts it:

```
COMMIT(QC) -- QC = QuorumCertificate { node_hash, view, signatures }
```

Upon receiving a COMMIT message, each node (including Observers) verifies the QC via `QuorumCertificate::is_valid` and, if valid, marks N as committed in its local replica. The committed node is then available to subsequent proposers as a valid parent for future nodes.

Message Sequence and Timing Bounds

Step	Message	Sender	Receivers	Bound (after GST)
1	PROPOSE	Proposer	All Validators	Δ
2	VOTE	Each Validator	Aggregator	2Δ
3	COMMIT	Aggregator	All Nodes	3Δ

Total latency from PROPOSE to COMMIT is 3Δ after GST. This is optimal for a three-phase protocol under partial synchrony.

5.4 View Change

If no COMMIT message is received within a timeout $T = 4\Delta$ of the PROPOSE, validators initiate a view change. The view change protocol proceeds as follows:

11. Each validator broadcasts `VIEW-CHANGE( $view+1$ ,  $highest\_pending\_hash$ ,  $\sigma$ )` where $highest_pending_hash$ is the hash of

the highest-height pending node the validator has seen in the expiring view.

12. Once $\lceil (2n+1)/3 \rceil$ VIEW-CHANGE messages for view $view+1$ are collected, the new Proposer (selected by round-robin or Verifiable Random Function (VRF) over the validator set) broadcasts NEW-VIEW($view+1$, *highest_pending_hash*).

13. Validators reset their timeout and continue protocol execution in view $view+1$, re-proposing the highest-priority pending node from the previous view.

Liveness is maintained because after GST, all honest validators receive the same VIEW-CHANGE messages within Δ and agree on the new Proposer and highest pending node within 2Δ . The new Proposer then proceeds to Phase 1 of AETHEL-BFT in the new view, and Phase 3 completes within 3Δ of the new PROPOSE.

5.5 DAG Sync Protocol

Nodes that have been offline or that receive a COMMIT referencing a node hash they do not possess use the DAG Sync Protocol to reconstruct their local DAG replica:

14. **Hash request:** The node broadcasts GET(hash) to its peers for each missing node hash encountered in parent references.

15. **Node response:** Any peer possessing the node responds with the full `CasNode` bytes.

16. **Recursive resolution:** For each received node, the recipient checks that all parent hashes are locally present, issuing GET requests for any missing ancestors. This process terminates because the DAG is acyclic (Theorem 2) and finite, with genesis as the unique root.

17. **Gossip propagation:** Nodes periodically broadcast their current tip hashes (the set of nodes with no children) to peers, enabling rapid discovery of new committed nodes without requiring a central coordinator.
18. **Topological replay:** Once all missing nodes are received and verified, the syncing node replays the DAG from genesis using `topological_iter` (Theorem 6), reconstructing all intermediate kernel states deterministically.

5.6 Sharding Extension (SHACASDAG-S)

SHACASDAG-S extends the base architecture with horizontal sharding for scalability beyond the capacity of a single validator set.

Each shard s_i maintains its own sub-DAG $G_i = (V_i, E_i)$, rooted at a shard-specific genesis node G_{i0} that references the global genesis node G_0 . Each shard has its own validator set and runs AETHEL-BFT independently for intra-shard consensus.

Cross-Shard Certificates (XSC): When a state transition requires coordination across two shards s_i and s_j , a Cross-Shard Certificate is constructed as a QC over the tuple $(\text{tip}(G_i), \text{tip}(G_j))$, signed by a cross-shard committee drawn from both validator sets. The cross-shard committee size satisfies the same $\lfloor (2n+1)/3 \rfloor$ threshold with respect to the combined validator population.

Global Invariant: The constitutional invariant is extended to a global predicate $I_{kl\emptyset=1}$ over all shard sub-DAGs: $I_{kl\emptyset=1} = \bigwedge_i I(G_i) \wedge \text{XSC-Consistency}$. XSC-Consistency asserts that all cross-shard certificates reference tip hashes that are committed in their respective shard DAGs at the time the XSC is assembled.

Shard routing is determined by a fixed partition function on the payload key space, registered in the genesis configuration. Shard membership is itself a first-class DAG node type, enabling the number of shards to grow via governance proposals following the same gate admission path as all other transitions.

6. Constitutional Governance Model

6.1 Gate Registry

The gate set Γ^* is established at kernel genesis via a signed gate manifest. The manifest is itself a `CasNode` of `NodeType = Genesis`, whose payload is the canonical encoding of the ordered list of gate specifications (gate ID, precondition hash, postcond hash, effect hash, activation view). The manifest's content address serves as the unique identifier of the kernel's constitutional configuration.

Adding a new gate to Γ^* requires a **governance proposal node** of `NodeType = GovernanceProposal`, whose payload encodes the proposed gate specification. This governance proposal node must itself pass all existing gates in Γ^* , ensuring that no new gate can be admitted without satisfying the currently operative constitutional invariants. This self-referential structure prevents bootstrapping attacks: an attacker cannot register a gate that bypasses existing invariants, because such a gate's admission proposal would be rejected by those same invariants.

Gate retirement follows the identical governance path. A gate is retired by a `GovernanceProposal` node encoding a gate removal operation, which must pass all remaining active gates. Importantly, a gate cannot retire itself: the precondition for a gate removal proposal must be evaluated under the full current gate set including the gate to be retired, preventing self-removal exploits.

6.2 Invariant Upgrades

The constitutional invariant I is represented as a composable predicate tree, serialized as a CAS node and referenced by hash from the gate manifest. This design affords the following upgrade properties:

- **Versioning:** Each version of I is a distinct CAS node with its own content address. The history of invariant versions is itself a DAG path within G .
- **Backwards compatibility:** Any proposed new invariant I' must satisfy $I'(v) \Rightarrow I(v)$ for all $v \in \text{committed}$, i.e., the new invariant must be at least as strong as the old on all existing committed nodes. This ensures that upgrading the invariant cannot retroactively invalidate the existing DAG history.
- **Predicate tree composition:** New atomic predicates are conjoined with existing predicates using a predicate tree combinator node, enabling additive invariant strengthening without replacing the entire predicate structure.
- **Audit trail:** Because the invariant is CAS-addressed and referenced from governance proposal nodes, the full history of constitutional changes is permanently and immutably recorded in the DAG.

6.3 Validator Set Management

Changes to the validator set are treated as first-class DAG transitions, encoded as CAS nodes of `NodeType = ValidatorSetChange`. A `ValidatorSetChange` node specifies:

- The set of public keys to add to the validator set;
- The set of public keys to remove from the validator set;
- The view number at which the change takes effect;

- A QC from the *current* validator set attesting to the validity of the change.

The requirement that a `ValidatorSetChange` node carries a QC from the current validator set provides the critical safety property: the existing quorum must authorize any membership change before it takes effect. This prevents a Byzantine coalition from unilaterally bootstrapping a new validator set that would have a different quorum composition.

Long-range attack mitigation: Validator set changes that occurred more than a configurable number of views in the past are anchored by checkpoint nodes (`NodeType = Checkpoint`), which are committed DAG nodes carrying a QC over a snapshot of the full kernel state at that height. New participants bootstrapping from genesis use checkpoint anchors to establish trust without replaying the full DAG history, mitigating long-range validator set manipulation attacks.

Rotation safety: The protocol guarantees that at no point does the validator set drop below the minimum quorum threshold. A `ValidatorSetChange` that would reduce the validator count below $3f+1$ (where f is the target fault tolerance) is rejected by the constitutional gate set as a precondition violation.

7. Security Analysis

7.1 Threat Model

We analyze the security of SHACASDAG against the following threat classes:

Threat	Description	Mitigation
Byzantine Validators	Up to $f < n/3$ validators deviate arbitrarily: equivocate, vote for	Quorum intersection (Theorem 4, Corollary 4.1); honest quorum

Threat	Description	Mitigation
	invalid nodes, or withhold votes.	required for any QC.
Network Partition	Honest validators are partitioned into disjoint groups unable to communicate.	Safety maintained unconditionally; liveness resumes after GST via view change (Section 5.4).
Replay Attacks	An attacker replays a previously committed node or QC to attempt double-commit or state regression.	Node hashes are content-addressed; a replayed node with the same hash is idempotent (already committed). View numbers in QCs prevent cross-view replay.
Hash Collision Attacks	An attacker attempts to produce two distinct payloads with the same SHA-256 hash.	Theorem 1; SHA-256 collision resistance provides 128-bit security against collision attacks.
Sybil Attacks	An attacker registers many pseudonymous validators to exceed the fault threshold.	Validator set is a first-class governance artifact; ValidatorSetChange requires QC from current set (Section 6.3).
Long-Range Attacks	An attacker with old validator keys attempts to rewrite ancient DAG history.	Checkpoint anchoring (Section 6.3); historical QCs are permanently committed and cannot be forged without re-obtaining quorum from the then-current validator set.

7.2 SHA-256 Collision Resistance

SHA-256 provides a 256-bit output space and is standardized by NIST [9]. The best known collision attacks against SHA-256 require on the order of 2^{128} operations (birthday bound), which exceeds the practical computational capacity of any foreseeable adversary by many orders of magnitude. For the content-addressing use case in SHACASDAG, the relevant security notion is second-preimage resistance (given a known node N , find $N' \neq N$ with $\text{SHA-256}(N') = \text{SHA-256}(N)$), for which SHA-256 provides 256-bit security —

computationally indistinguishable from ideal for any polynomially bounded adversary.

Should SHA-256 be weakened by future cryptanalytic advances, SHACASDAG provides an upgrade path via the gate manifest: a new hash function (SHA-3-256 or BLAKE3) can be registered as a gate parameter, and a governance proposal can mandate that new nodes use the upgraded hash function from a specified view onward. Existing committed nodes retain their SHA-256 addresses permanently; only new nodes adopt the new function. This provides forward security without invalidating historical DAG entries.

7.3 Signature Scheme

Ed25519 [10] is selected as the primary signature scheme for the reference implementation. Ed25519 provides 128-bit security against all known attacks, produces 64-byte signatures, and achieves signature verification throughput of several hundred thousand operations per second on commodity hardware — well within the performance requirements of AETHEL-BFT's VOTE phase.

As the validator set grows, the $O(n)$ signature aggregation cost of collecting individual Ed25519 signatures into a QC becomes the bottleneck. **BLS12-381** threshold signatures [13] are identified as the primary upgrade path: BLS signatures are homomorphically aggregable, reducing the QC to a single constant-size signature regardless of the number of validators, at the cost of higher per-operation latency and a pairing-based assumption. BLS12-381 integration is planned as a Phase 3+ enhancement, configurable via the hash and signature algorithm fields of the gate manifest.

7.4 Known Limitations

19. **Partial synchrony assumption:** Liveness (Theorem 4) is conditioned on eventual network synchrony (post-GST). In permanently asynchronous environments (e.g., extreme network partitions),

SHACASDAG preserves safety but cannot guarantee progress. This is a fundamental consequence of the FLP impossibility result and is not specific to SHACASDAG.

20. **Long-range attacks at genesis:** If the initial validator set is compromised at genesis (before any checkpoints are established), an attacker could rewrite the entire DAG history. This is mitigated by out-of-band genesis configuration distribution and by anchoring the genesis hash in an external trust anchor (e.g., a public ledger or notary service).
21. **Formal verification gap:** The Rust implementation has not yet been formally verified against the TLA+ specification. Until the Prusti [see roadmap] annotation pass is complete (Phase 5), the implementation's correctness relative to the formal model rests on testing and code review rather than machine-checked proof. This is the highest-priority risk item in the project roadmap.
22. **Canonical encoding determinism:** The correctness of hash-based content addressing depends on `canonical_encode` being strictly deterministic. Any serialization non-determinism (e.g., map key ordering, floating-point representation) would cause nodes constructed identically on different machines to have different hashes, breaking DAG sync. The bincode 2.0 schema-pinned configuration eliminates this risk for the reference implementation, but alternative implementations must adhere to the same encoding specification.

8. Implementation Roadmap

Development is organized into six sequential phases, each delivering a self-contained, testable milestone. Phase boundaries are aligned with AETHEL Systems Research's quarterly planning cycle.

Phase	Deliverable	Key Artifacts	Target
0 — Genesis	Core CAS node + SHA-256 verification + single-node kernel	CasNode, DagStore (single-node), KernelError, genesis config, unit test suite	Q3 2026
1 — Gates	Gate trait + AETHEL Gate set + invariant checking	Gate trait, gate manifest, constitutional invariant evaluator, gate integration tests, TLA+ draft spec	Q4 2026
2 — Consensus	AETHEL-BFT 3-phase protocol + QC aggregation	QuorumCertificate, PROPOSE/VOTE/COMMIT message types, Aggregator service, consensus integration tests	Q1 2027
3 — Multinode	Full n -validator network + DAG sync + view change	gRPC peer protocol (tonic), DAG sync service, view change handler, 5-node testnet, chaos test suite	Q2 2027
4 — Sharding	SHACASDAG-S cross-shard extension	Shard sub-DAG, XSC protocol, cross-shard committee selection, global invariant evaluator, multi-shard testnet	Q3 2027
5 — Formal	TLA+ model check completion + Prusti annotation pass	TLC/Alpache model check results, Prusti-annotated kernel core, differential testing harness, security audit report	Q4 2027

Each phase is gated by the successful completion of its predecessor's test suite and a documented architecture review. Phase 5 in particular is a prerequisite for any production deployment recommendation; until formal verification is complete, SHACASDAG is classified as research-grade software.

9. Conclusion

We have presented SHACASDAG, a SHA-Anchored Content-Addressable Storage / Directed Acyclic Graph Constitutional Kernel, offering a formally grounded architecture for invariant-preserving distributed state machines. The core insight of SHACASDAG is that the three pillars of distributed systems correctness — content integrity, causal ordering, and invariant preservation — are naturally unified by treating every state transition as a SHA-256-hashed CAS node in a Directed Acyclic Graph, admissible only through formally specified AETHEL Gates whose predicates are expressed in TLA+.

The theorem hierarchy established in Section 2 proves, under standard cryptographic assumptions and the partial synchrony network model, that SHACASDAG achieves: SHA integrity (Theorem 1), DAG acyclicity preservation (Theorem 2), constitutional invariant preservation across all admitted transitions (Theorem 3), gate soundness (Lemma 3.1), liveness under Byzantine-fault-tolerant quorum (Theorem 4), fork safety (Corollary 4.1), kernel minimality (Theorem 5), and deterministic replay from genesis (Theorem 6). These theorems collectively establish that the constitutional kernel achieves its three design objectives by construction rather than by convention.

The TLA+ specification presented in Section 3 provides a machine-checkable formal model of the AETHEL Gate Pipeline, directly linking the informal theorem statements to a model-checkable artifact. The Rust reference implementation in Section 4 realizes this formal model in a production-grade codebase, with a no-unsafe kernel core, trait-based gate extensibility, RocksDB-backed persistent storage, and Ed25519 signature verification. The multinode extension in Section 5 demonstrates that SHACASDAG's guarantees scale to Byzantine-fault-tolerant distributed deployments via the AETHEL-BFT three-phase protocol, with a clear upgrade path to horizontal sharding via SHACASDAG-S.

The governance model in Section 6 ensures that the kernel's constitutional structure is itself subject to constitutional discipline: gates, invariants, and validator sets can only evolve through the same gate-admission mechanism they govern, providing a self-consistent and auditable governance record permanently encoded in the DAG.

The most significant near-term priorities are the completion of the TLA+ model check (verifying the formal specification against finite instantiations with TLC and Apalache), the Prusti annotation pass on the Rust kernel core (closing the gap between the formal model and the implementation), and the BLS12-381 signature aggregation integration (enabling scalable QC construction for large validator sets). Upon completion of Phase 5, SHACASDAG is expected to provide the strongest formal correctness guarantees of any distributed constitutional kernel in the open research literature.

SHACASDAG is developed under the AETHEL Systems Research program. Specifications, implementation source, and TLA+ models will be released as open research artifacts upon Phase 1 completion.

References

23. Lamport, L. (1994). *The Temporal Logic of Actions*. ACM Transactions on Programming Languages and Systems, 16(3), 872–923. The foundational paper establishing TLA+ as a specification language for concurrent and distributed systems.
24. Castro, M., & Liskov, B. (1999). *Practical Byzantine Fault Tolerance*. In Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI), pp. 173–186. USENIX. The seminal paper establishing the PBFT three-phase protocol and quorum intersection arguments underpinning AETHEL-BFT.
25. Torvalds, L., et al. (2005). *Git Distributed Version Control System*. Technical design: Merkle DAG of content-addressed commit objects with hash-linked parent references. Available at: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
26. Benet, J. (2014). *IPFS — Content Addressed, Versioned, P2P File System*. arXiv preprint arXiv:1407.3561. Establishes the practical architecture for SHA-addressed content-addressable storage in a distributed peer-to-peer network.
27. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., & Winwood, S. (2009). *seL4: Formal Verification of an OS Kernel*. In Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), pp. 207–220. The existence proof for formally verified production kernel deployment.
28. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Whitepaper. The original presentation of the proof-of-work blockchain

as a hash-linked chain with Byzantine fault tolerance via computational consensus.

29. Popov, S. (2018). *The Tangle*. IOTA Foundation Whitepaper, Version 1.4.3. Describes a DAG-structured distributed ledger (the Tangle) in which each transaction directly approves two prior transactions, providing a reference for DAG-based consensus designs.
30. Rocket, T., Yin, M., Sekniqi, K., van Renesse, R., & Sirer, E. G. (2019). *Scalable and Probabilistic Leaderless BFT Consensus through Metastability*. arXiv preprint arXiv:1906.08936. Describes the Avalanche DAG consensus family, providing a reference for probabilistic finality in DAG-structured distributed systems.
31. National Institute of Standards and Technology (NIST). (2015). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce. The normative standard specifying SHA-256 and the SHA-2 family of hash functions.
32. Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). *High-Speed High-Security Signatures*. Journal of Cryptographic Engineering, 2(2), 77–89. The original presentation of Ed25519, establishing its 128-bit security level, performance characteristics, and batch verification properties.
33. Jung, R., Jourdan, J.-H., Krebbers, R., & Dreyer, D. (2021). *RustBelt: Securing the Foundations of the Rust Programming Language*. Journal of the ACM (JACM), 68(4), 1–34. Formal proof that the Rust type system — including ownership, borrowing, and lifetimes — provides memory safety and data-race freedom for safe code.
34. Dong, S., Callaghan, M., Galanis, L., Borthakur, D., Savor, T., & Strum, M. (2017). *Optimizing Space Amplification in RocksDB*. In Proceedings of the 8th CIDR Conference. Describes the RocksDB

LSM-tree architecture used for persistent DAG node storage in the SHACASDAG reference implementation.

35. Boneh, D., Drijvers, M., & Neven, G. (2018). *Compact Multi-Signatures for Smaller Blockchains*. In Advances in Cryptology — ASIACRYPT 2018, pp. 435–464. Springer. Describes BLS12-381 threshold signature aggregation, the target upgrade path for SHACASDAG QC size reduction.
36. Dwork, C., Lynch, N., & Stockmeyer, L. (1988). *Consensus in the Presence of Partial Synchrony*. Journal of the ACM (JACM), 35(2), 288–323. Establishes the partial synchrony network model and demonstrates that safety and liveness are achievable under eventual synchrony, underpinning the AETHEL-BFT liveness analysis.
37. Bertot, Y., & Castéran, P. (2004). *Interactive Theorem Proving and Program Development: Coq'Art — The Calculus of Inductive Constructions*. Springer. The primary reference for the Coq proof assistant, identified as an alternative mechanization target for SHACASDAG theorem proofs alongside TLA+.

SHACASDAG Whitepaper — Version 1.0-DRAFT — July 2026 — AETHEL
Systems Research

Author: John — Bethel Acres, OK, United States

This document is a draft specification. All theorem proofs are sketches
pending full mechanization in Phase 5.